

Projet p_WEB295

Passion Lecture API REST

Stepan et Camille

28/04/2026

TABLE DES MATIERES

Introduction et analyse.....	2
Compréhension du cahier des charges.....	3
Planification.....	4
Utilisation de GitHub.....	5
MODEL de base de données.....	5
Maquette et prototype.....	7
MISE EN PLACE D'adonis et ORM Lucid	7
Pourquoi un ORM plutôt que du SQL ?.....	8
Réalisation.....	8
Migrations et modèles.....	9
Migrations	9
Modèles Lucid.....	9
Seeders et Factories.....	9
Validation des données avec VineJS.....	10
Pagination, tri et filtrage des recherche.....	11
CORS.....	12
RUNNER Bruno	13
Tests effectués.....	14
Connexion du frontend au backend.....	15
Conclusion	16
Conclusion personnelle Stepan.....	17
Conclusion personnelle Camille.....	17
Critique constructive.....	18
UTILISATION DE L'IA.....	19
Camille:	19
Stepan :.....	19

INTRODUCTION ET ANALYSE

Dans le cadre du module 295, nous avons réalisé le backend d'une application web appelée Passion Lecture ou nous avons déjà réalisé le frontend dans le module 294.

Notre objectif était de construire une API REST complète qui sera utilisée par un frontend Vue.js développé lors d'un module précédent.

Pour cela, nous avons utilisé AdonisJS, un framework Node.js orienté backend.

La base de données est MySQL, et nous avons utilisé l'ORM Lucid fourni par AdonisJS pour faire le lien entre notre code et la base.

L'API permet à un utilisateur final, à travers le frontend, de gérer des livres, leurs auteurs, leurs catégories, les commentaires postés sur les livres, ainsi que les comptes utilisateurs.

Elle gère l'authentification, les droits d'accès, la validation des données et la pagination des résultats.

COMPRÉHENSION DU CAHIER DES CHARGES

Le cahier des charges demandait de mettre en place une API REST avec :

- La gestion CRUD
 - a. Créer
 - b. Lire
 - c. Modifier
 - d. Supprimer
- La validation de toutes les données envoyées par le client
- La gestion correcte des codes HTTP
 - a. 200/201 pour succès
 - b. 4... pour erreurs client
 - c. 5... pour erreurs serveur
- La recherche, le tri et le filtrage sur les livres et autres ressources
- L'authentification par jetons OAT
 - a. Opaque Access Tokens
- Une documentation Swagger de toutes les routes
- Des tests de l'API avec Bruno
- La gestion des règles CORS pour communiquer avec le frontend

L'objectif final était de livrer une API fonctionnelle, sécurisée et bien documentée, directement utilisable par le frontend Vue.js sur lequel nous avons travaillé précédemment.

PLANIFICATION

Pour planifier ce projet et nous partager les tâches nous avons utilisé une méthode proche de scrum.

Ensemble nous avons fait le point sur le cahier des charges, nous avons listé dans un projet GitHub les fonctionnalités à implémenter.

Pour travailler nous avons fait le point à chaque début de cours, qui fait quoi aujourd'hui, parlé de ce qui a été fait et regarder le code ensemble.

Nous avons parfois pris le temps durant les sessions de travail pour regarder le code ensemble, et faire le point quand cela était nécessaire, ou simplement pour quittancer une tâche finie avant de passer à la prochaine.

Ainsi tout au long du projet nous savions ou nous en étions, personne n'est resté sur des blocages et nous étions toujours au courant de ce que faisait l'autre.

Nous avons pu avancer ensemble la plupart du temps.

UTILISATION DE GITHUB

Nous avons utilisé Git et GitHub pour gérer les versions du code et permettre un travail à deux propre et organisé. Chaque fonctionnalité importante était développée dans une branche séparée, puis intégrée à la branche principale via un merge une fois testée.

GitHub nous a aussi servi pour suivre les tâches grâce aux issues créées à partir du cahier des charges.

Chaque issue représentait une fonctionnalité.

Cela permettait à chacun de voir clairement ce qu'il restait à faire et ce qui était terminé.

GitHub nous a permis de suivre les tâches grâce aux issues créées à partir du CDC, et de garder une organisation claire dans les projets via les commits.

MODEL DE BASE DE DONNÉES

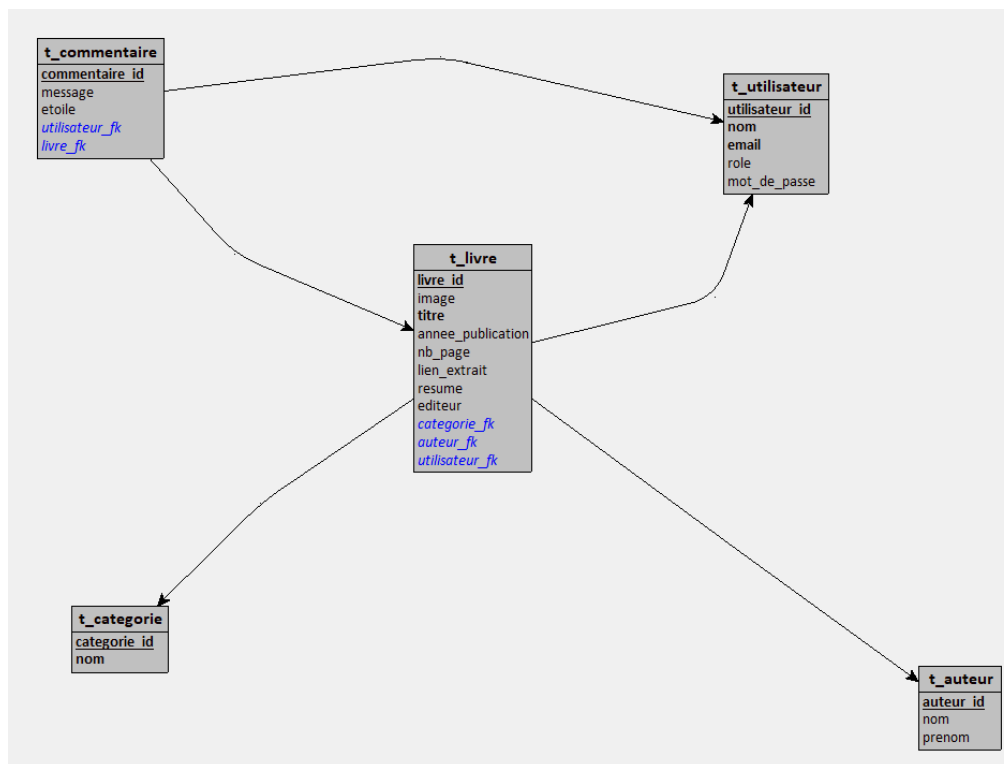
Avant de coder, nous avons créé un MCD et un MLD pour définir les relations entre les tables et la structure générale de l'application.

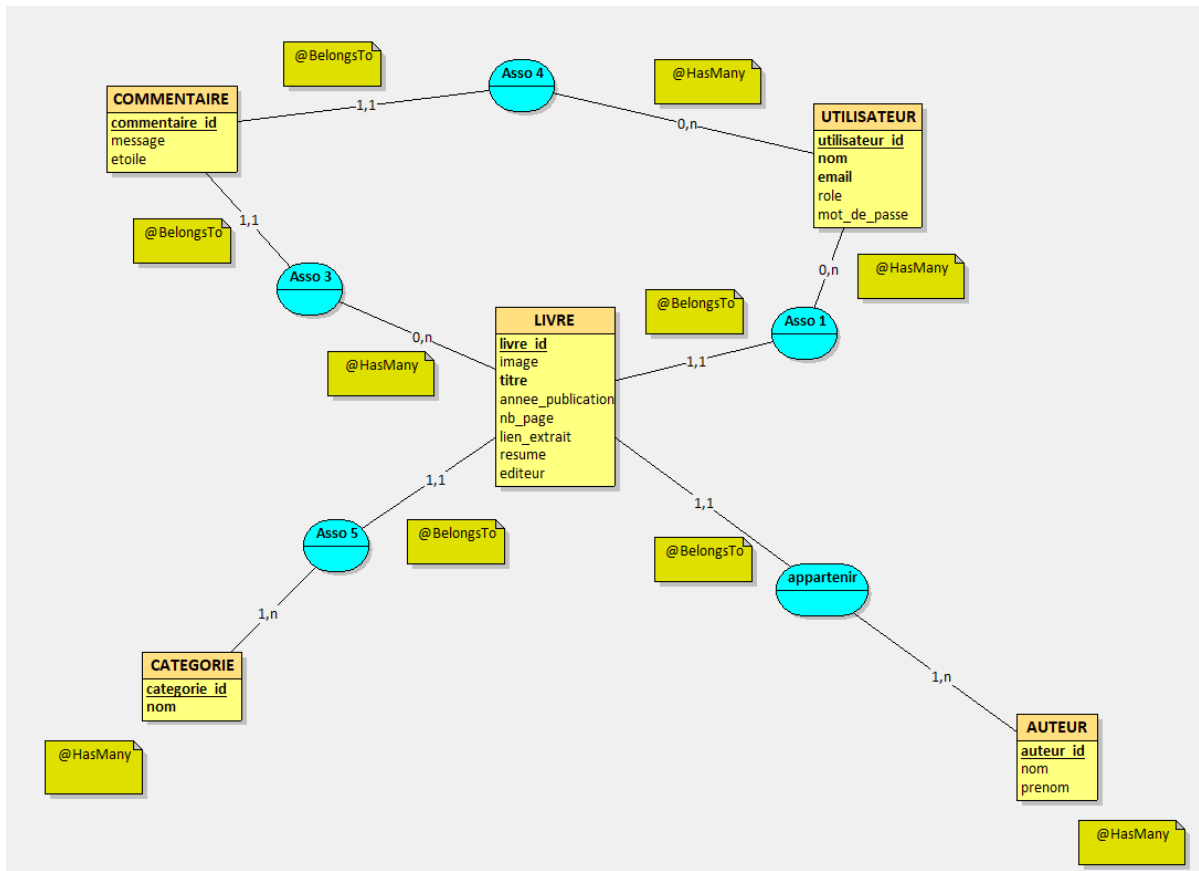
Les principales tables sont :

- books : id, titre, image, annee_publication, nb_page, lien_extrait, resume, editeur, category_id, author_id, user_id
- authors : id, last_name, first_name
- categories : id, name
- users : id, full_name, email, role, password
- comments : id, message, etoile, user_id, book_id

Les relations principales sont :

- Un livre appartient à un auteur, une catégorie et un utilisateur qui l'a ajouté
- Un commentaire appartient à un utilisateur et à un livre
- Un utilisateur peut avoir plusieurs commentaires et plusieurs livres





MAQUETTE ET PROTOTYPE

Bien que la maquette ait été réalisée durant le module précédent, nous l'avons gardée comme référence visuelle pour savoir quelles données l'API devrait fournir. La maquette est disponible sur Figma :

https://www.figma.com/design/6nTOrngEsmMletrIqnTm3/Mockup_PassionLecture

MISE EN PLACE D'ADONIS ET ORM LUCID

AdonisJS est un framework Node.js qui propose une structure complète pour développer une API.

Il inclut son propre ORM appelé Lucid, qui permet de manipuler la base de données en TypeScript plutôt qu'en SQL brut.

Pour connecter AdonisJS à notre base MySQL, nous avons configuré le fichier .env avec les variables suivantes : host, port, nom de la base, utilisateur et mot de passe. Adonis lit automatiquement ce fichier au démarrage.

Nous avons ensuite créé les migrations pour générer les tables, les modèles Lucid pour représenter chaque entité dans le code, et les seeders pour peupler la base avec des données de test générées via Faker.js.

POURQUOI UN ORM PLUTÔT QUE DU SQL ?

Un ORM (Object-Relational Mapping) permet de travailler avec des objets TypeScript au lieu d'écrire des requêtes SQL à la main.

Par exemple, pour récupérer un livre avec son auteur, on écrit :

```
const book = await Book.query().preload('author').firstOrFail()
```

Au lieu d'écrire une jointure SQL manuellement.

Cela rend le code plus court, plus lisible, et évite les fautes de frappe SQL.

Lucid s'occupe aussi de convertir automatiquement les noms : `userId` en TypeScript devient `user_id` en base de données.

MIGRATIONS ET MODÈLES

MIGRATIONS

Les migrations sont des fichiers TypeScript qui décrivent la structure des tables.

Chaque migration possède une méthode `up()` qui crée la table et une méthode `down()` qui la supprime.

Cela permet de versionner le schéma de la base au même titre que le code.

Une migration importante a été celle de la table `books`.

Nous avons défini les clés étrangères vers `categories`, `authors` et `users`, avec `onDelete('CASCADE')` pour que les livres soient supprimés automatiquement si leur catégorie ou auteur est supprimé.

MODÈLES LUCID

Les modèles représentent chaque entité de la base dans notre code.

Ils utilisent des décorateurs comme `@column`, `@belongsTo`, `@hasMany` pour décrire les colonnes et les relations.

Exemple simplifié du modèle `Book` :

```
@column() declare titre: string
@belongsTo(() => Author) declare author: BelongsTo<typeof Author>
@hasMany(() => Comment) declare comments: HasMany<typeof Comment>
```

SEEDERS ET FACTORIES

Pour tester l'API avec des données réalistes, nous avons créé des factories avec Faker.js et des seeders. Plusieurs problèmes ont été rencontrés et résolus :

- Oubli du mot-clé `await` dans les seeders, ce qui causait l'insertion sans attendre la fin
 - a. Corrigé en ajoutant `await` partout
- Erreur 'Cannot get value from empty dataset' car `book_seeder` tournait avant `category_seeder` (ordre alphabétique)
 - a. Résolu en créant un `main_seeder` qui appelle les seeders dans le bon ordre
- Erreur 'Cannot define category_id on Book model' car les clés étrangères n'étaient pas déclarées dans le modèle
 - a. Résolu en ajoutant `@column()` pour `categoryId`, `authorId` et `userId`
- Erreur de référence circulaire JSON car `faker.string` et `faker.book.author` étaient passés sans `()`
 - a. Corrigé en appelant `faker.string.uuid()` et `faker.company.name()`

La commande utilisée pour réinitialiser et repeupler la base est :

```
node ace migration:fresh
```

Puis

```
node ace db:seed --files='database/seeder/main_seeder.ts'
```

Car un fichier `main_seeder.ts` a été créé pour résoudre les problèmes d'ordre d'exécution des seeders. C'était la solution la plus simple pour ne pas perdre trop de temps.

Ce fichier force l'ordre d'exécution des seeders.

VALIDATION DES DONNÉES AVEC VINEJS

Toutes les données envoyées par le client doivent être validées avant d'être enregistrées en base.

Sans validation, un utilisateur malveillant pourrait envoyer n'importe quoi et faire planter le serveur, par exemple un titre vide, un nombre de pages négatif, ou des injections SQL.

Pour cela, nous avons utilisé VineJS, la bibliothèque de validation intégrée à AdonisJS.

Chaque ressource a son propre fichier de validateur dans le dossier `app/validators/`.

Exemple du validateur pour créer un livre :

```
titre: vine.string().minLength(1).maxLength(255),  
nb_page: vine.number().min(1).max(9999),  
annee_publication: vine.number().min(1000).max(9999),
```

Quand le client envoie une donnée invalide, AdonisJS renvoie automatiquement une erreur HTTP 422 (Unprocessable Entity) avec un message détaillé.

Cela permet au frontend d'afficher facilement l'erreur à l'utilisateur.

Nous avons aussi créé des validateurs séparés pour la modification (`updateBookValidator`), où tous les champs sont optionnels, afin de pouvoir mettre à jour seulement une partie d'un livre sans devoir renvoyer toutes les données.

PAGINATION, TRI ET FILTRAGE DES RECHERCHE

Sans pagination, une route comme `GET /books` pourrait retourner des milliers de livres d'un seul coup.

Cela alourdirait la réponse HTTP, ralentirait l'affichage côté client et serait inutilisable en pratique.

Nous avons donc mis en place une pagination sur toutes les routes index, en respectant le format AdonisJS natif.

Les paramètres de pagination passent par l'URL en query params :

```
GET  
/books?page=1&limit=10&sort=titre&order=asc&search=harry&categoryId=2
```

La réponse JSON contient deux blocs :

- meta : informations de pagination (total d'éléments, page courante, dernière page, URL suivante, etc.)
- data : tableau des éléments de la page demandée

Pour valider les paramètres reçus, nous avons créé un validateur dédié (book_query_validator.ts) qui contrôle que page, limit, sort et order ont des valeurs autorisées.

Par exemple, sort ne peut prendre que les valeurs « titre », « nb_page », « annee_publication » ou « created_at ».

Cela évite qu'un utilisateur malveillant essaie de trier par un champ sensible comme « password ».

Le tri, le filtrage et la recherche sont combinables : on peut demander la deuxième page des livres de la catégorie 2, triés par titre dans l'ordre alphabétique, qui contiennent le mot « harry » dans leur titre, éditeur ou résumé.

CORS

Pour que le frontend Vue.js puisse communiquer avec notre API, nous avons dû configurer les règles CORS : Cross-Origin Resource Sharing.

Sans cette configuration, le navigateur bloque automatiquement les requêtes venant d'un domaine différent, ce qui empêcherait le frontend de parler au backend.

Dans AdonisJS, la configuration CORS se trouve dans le fichier : config/cors.ts.

Nous avons autorisé les requêtes venant de <http://localhost:5173> qui est l'adresse du frontend Vue.js en développement, et activé les méthodes HTTP nécessaires : GET, POST, PUT et DELETE.

Nous avons aussi activé `credentials: true` pour que les tokens d'authentification puissent être transmis correctement.

RUNNER BRUNO

Pour tester l'API de manière complète et dans le bon ordre, nous avons utilisé le Runner de Bruno.

Cet outil permet d'exécuter toutes les requêtes d'une collection automatiquement, l'une après l'autre.

L'ordre est important car certaines requêtes dépendent des résultats des précédentes. Par exemple, il faut d'abord créer un compte et se connecter pour obtenir le token, avant de pouvoir créer un livre.

L'ordre utilisé était le suivant : register, login, puis les opérations sur les utilisateurs, les auteurs, les catégories, les livres et enfin les commentaires. La suppression des données se fait en dernier, et le logout termine la séquence.

La requête de login contient un script qui sauvegarde automatiquement le token dans les variables d'environnement Bruno, ce qui permet aux requêtes suivantes de l'utiliser sans le copier-coller manuellement.

Voici l'ordre dans le quelle a été fait le test du Runner dans Bruno :

- ✓ users\Se register 201 - Created
- ✓ users\Se login 200 - OK
- ✓ users\obtenir tout les users 401 - Unauthorized
- ✓ users\obtenir 1 user 401 - Unauthorized
- ✓ users\modifier un user 401 - Unauthorized
- ✓ authors\obtenir tout les auteurs 200 - OK
- ✓ authors\créer 1 auteur 422 - Unprocessable Entity
- ✓ authors\obtenir 1 auteur 200 - OK
- ✓ authors\modifier 1 auteur 500 - Internal Server Error
- ✓ categories\recupérer toutes les catégories 200 - OK
- ✓ categories\créer 1 catégorie 500 - Internal Server Error
- ✓ categories\recupérer 1 categorie 200 - OK
- ✓ categories\mettre a jour une catégorie 200 - OK
- ✓ books\recupère tout les livres 200 - OK
- ✓ books\créer 1 livre 401 - Unauthorized
- ✓ books\recupère 1 livre 200 - OK
- ✓ books\update de 1 livre 401 - Unauthorized
- ✓ commentaires\Recup tout les coms 200 - OK
- ✓ commentaires\Créer un com 404 - Not Found
- ✓ commentaires\Recup un com 200 - OK
- ✓ commentaires\del un com 404 - Not Found
- ✓ books\supprimer 1 livre 401 - Unauthorized
- ✓ authors\supprimer 1 auteur 200 - OK
- ✓ categories\supprimer une catégorie 200 - OK
- ✓ users\supprimer un user 401 - Unauthorized
- ✓ users\Se logout 401 - Unauthorized

TESTS EFFECTUÉS

Pour tester notre API, nous avons utilisé Bruno, un logiciel qui permet d'envoyer des requêtes HTTP directement à notre backend et de vérifier les réponses.

Nous avons créé une collection de requêtes organisées par ressource : utilisateurs, auteurs, catégories, livres et commentaires.

Chaque requête correspond à une action précise de l'API.

Pour tester dans le bon ordre, nous avons utilisé le Runner de Bruno qui exécute toutes les requêtes automatiquement à la suite.

L'ordre est important car certaines requêtes dépendent des résultats des précédentes. Par exemple, il faut obligatoirement s'enregistrer et se connecter avant de pouvoir créer un livre, car les routes protégées exigent un token d'authentification.

La requête de login contient un petit script qui sauvegarde automatiquement le token reçu dans les variables d'environnement de Bruno.

Toutes les requêtes suivantes utilisent ce token via la variable `{{token}}`, sans avoir à le copier-coller manuellement.

L'ordre des tests dans le Runner était le suivant :

1. Register (créer un compte)
2. Login (récupère et sauvegarde le token)
3. Opérations sur les utilisateurs (lire, modifier)
4. Créer un auteur, puis le lire et le modifier
5. Créer une catégorie, puis la lire et la modifier
6. Créer un livre (utilise l'auteur et la catégorie créés avant), puis le lire et le modifier
7. Créer un commentaire sur le livre, puis le lire
8. Supprimer les données dans l'ordre inverse (commentaire, livre, catégorie, auteur, utilisateur)
9. Logout

Toutes les requêtes ont retourné un code HTTP 200 OK ou 201 Created selon l'action effectuée, ce qui confirme que l'API fonctionne correctement.

CONNEXION DU FRONTEND AU BACKEND

Nous avons dû connecter le frontend Vue.js au backend AdonisJS.

Le frontend avait été développé lors du module précédent avec JSON Server : un faux serveur local qui simulait une API.

Tous les appels étaient donc écrits pour fonctionner avec JSON Server, ce qui voulait dire des noms de champs différents, des URLs différentes et aucune gestion de token.

Pour connecter les deux, nous avons dû réécrire les fichiers qui s'occupent des appels HTTP. Nous avons créé des fichiers dans un dossier, par exemple : `api/` : `api_book.js` et `api_comment.js`.

Ces fichiers contiennent des fonctions qui envoient les requêtes à notre API AdonisJS avec les bons noms de champs.

Nous avons aussi dû gérer l'authentification côté frontend

Après le login, l'API renvoie un token que l'on sauvegarde dans le `localStorage` du navigateur.

Ensuite, chaque requête qui nécessite d'être connecté envoie ce token dans le header HTTP sous la forme `Authorization: Bearer <token>`. Sans ce token, l'API refuse la requête avec un code 401.

Le logout supprime le token du `localStorage` et cela déconnecte l'utilisateur côté frontend.

CONCLUSION

Ce projet nous a permis de découvrir concrètement comment fonctionne une API REST complète, de la base de données jusqu'aux requêtes HTTP.

Nous avons compris les difficultés de faire le front entièrement et ensuite de créer le back puis de les connecter ensemble.

Nous avons appris à structurer un projet backend avec AdonisJS, à gérer les relations entre les tables, à valider les données et à sécuriser les routes avec un système d'authentification par token.

On retient que travailler à deux sur un même projet technique, du front au backend demande de la communication et de la rigueur. Cela demande de la communication, de la rigueur dans le code et une bonne organisation.

CONCLUSION PERSONNELLE STEPAN

En réalisant ce projet, j'ai pu approfondir mes connaissances en backend, un domaine que je maîtrisais moins bien que le frontend.

Il nous a aussi permis de mieux comprendre comment le frontend et le backend communiquent ensemble, ce qui m'aide maintenant à mieux voir l'application dans tout son ensemble et pas seulement du côté visuel.

La connexion entre le front et le back étaient vraiment intéressantes à savoir et les modifications à apporter avec l'ancien "backend" même si ce n'est pas un.

CONCLUSION PERSONNELLE CAMILLE

En réalisant ce projet, j'ai découvert la complexité de connecter un frontend et un backend ensemble. J'ai souvent eu du mal à visualiser l'ordre des étapes et les relations entre les différentes parties du code. La documentation reçue en cours a été indispensable.

Travailler à deux m'a parfois rendu difficile d'avoir une vision d'ensemble du projet, et j'ai ressenti une pression de ne pas ralentir mon collègue. Malgré cela, cette expérience m'a appris l'importance de la communication dans un projet technique partagé, et m'a montré les points sur lesquels je dois encore progresser.

CRITIQUE CONSTRUCTIVE

Si nous devons refaire ce projet, nous commencerions par mieux planifier la structure des validateurs et des modèles avant de coder, car nous avons perdu du temps à corriger des erreurs qui auraient pu être évitées dès le départ.

Aussi les relations où il faut bien faire dès le début et de y consacrer du temps et pas à remettre à plus tard sinon compliqué pour résoudre des problèmes.

Nous aurions aussi aimé avoir plus de temps pour tester davantage les cas limites dans Bruno, résoudre et comprendre nos problèmes avec le runner.

UTILISATION DE L'IA

CAMILLE:

J'ai utilisé Claude comme assistant pour le débogage des seeders, des migrations et la connexion du frontend au backend.

L'IA m'a permis de comprendre la logique derrière les solutions proposées avant de les appliquer et de les adapter à notre projet.

Chaque solution proposée a été comprise, testée et adaptée, cela m'a surtout permis de bien comprendre comment adapter la doc du module à notre projet, car parfois je n'avais pas forcément d'erreur mais cela ne fonctionnait pas et je ne trouvais pas les fautes.

L'IA a aussi été utilisée comme outil de recherche et d'explication pour mieux comprendre certains concepts comme les middlewares et la pagination dans AdonisJS. Je me suis servie du code pour comprendre la logique avant de l'implémenter moi-même.

STEPAN :

J'ai utilisé l'IA pour approfondir quelques connaissances comme le fait qu'on peut cibler certaines routes http comme update et delete qui aient besoin de l'authentification.

J'ai aussi utilisé pour gérer le style et UX sur le front end, pour déboguer certaines erreurs